PrivAgent: Top 50 Technical Interview Questions & Answers

Author: Yash Thakre (Roll No: 23BTB0A33 | B.Tech, Minor in Management | NIT Warangal)
Target: Tier-1 Technical Placement Interviews (Google, Microsoft, Amazon, Top AI Startups)

Pillar 1: Core Architecture & LangGraph Design (Q1 - Q10)

Q1: What is PrivAgent and what core enterprise problem does it solve?

Answer: PrivAgent is an on-premises Enterprise AI Operating System built with LangGraph. It solves the problem of enterprise data being siloed across relational databases, PDFs, Jira tickets, and Git repos, which cannot be sent to public cloud AI APIs due to GDPR, SOC2, HIPAA, and IP leakage risks. It orchestrates specialized local agents to answer complex multi-source queries securely without cloud data egress.

Q2: Why did you build PrivAgent with LangGraph instead of standard LangChain sequential chains?

Answer: Standard LangChain chains are strictly linear (acyclic) and brittle—if one step fails, the entire workflow halts without recovery. LangGraph allows cyclic graphs, enabling iterative self-correction loops between the Analyst and Critic, dynamic replanning on sub-agent errors, PostgreSQL state checkpointing, and native Human-in-the-Loop graph interrupts.

Q3: What is the exact data structure of AgentState in src/state.py?

Answer: AgentState is a TypedDict containing: user_query (str), intent (dict), plan (list of task dicts), current_task_index (int), logs (list of log strings), final_response (str), errors (list of error strings), critic_feedback (str), user_role (str), approved (bool), selected_model (str), draft_response (str), retry_count (int), and disable_critic (bool).

Q4: How does the 3-Pathway intent router work in src/agents/router.py?

Answer: Pathway 1 (Fast-Track/Cache) bypasses planning for simple greetings or semantic memory cache hits (<50ms). Pathway 2 (Single Specialist) routes directly to one worker agent when only one domain is needed (e.g. pure SQL). Pathway 3 (Multi-Agent DAG) triggers the Planning Agent to decompose complex multi-source goals into a Directed Acyclic Graph.

Q5: What happens if an investigator agent fails or throws an exception during graph execution?

Answer: The node catches the exception and appends an error message to state['errors']. In src/graph.py, the execution_router checks state['errors']; if new errors exist, it redirects execution to planning_agent_node, which inspects the failed task and dynamically replans an alternate task or retry strategy.

Q6: How is the cyclic loop between Analyst Agent and Critic Agent implemented?

Answer: In src/graph.py, workflow.add_edge('analyst_agent', 'critic_agent') connects the nodes. The critic_router checks state['critic_feedback']. If it starts with 'PASS', it routes to 'final_answer'. If it flags errors and state['retry_count'] < 2, it increments the retry counter and loops back to 'analyst_agent' for draft refinement.

Q7: What prevents an infinite loop between the Analyst and Critic Agents?

Answer: The retry_count variable in AgentState. The critic_router strictly enforces that if retry_count reaches 2, the loop breaks and routes to final_answer regardless of feedback, preventing infinite recursion and bounding maximum latency.

Q8: What is the role of the log_groomer_node in src/agents/synthesizer.py?

Answer: In multi-step workflows, cumulative logs dumped by SQL tables, Jira tickets, and document snippets can exceed 4,000 tokens. The log_groomer_node intercepts the state between worker steps, detects entries > 1,000 characters, and truncates them (keeping head 500 and tail 200 chars while preserving JSON structures), cutting context size by 65%.

Q9: How does PostgresSaver handle state persistence in src/graph.py?

Answer: PostgresSaver connects to PostgreSQL (port 5432) and creates checkpoint tables (checkpoints, checkpoint_blobs, checkpoint_writes). It serializes binary state snapshots per thread_id after every node execution. If the database is unreachable, get_graph() catches the exception with a 3s timeout and falls back to in-memory MemorySaver.

Q10: Why did you use TypedDict instead of Pydantic BaseModel for AgentState?

Answer: LangGraph operates on dictionary state updates. Using TypedDict allows fast, in-place key updates without the CPU serialization and model-validation overhead that Pydantic enforces at every single node transition, significantly reducing graph traversal latency on local machines.

Pillar 2: RAG, Vector DB & Information Retrieval (Q11 - Q20)

Q11: What is the exact formula for Hybrid RAG fusion in docs_agent_node?

Answer: HybridScore = (0.7 * VectorCosineSimilarity) + (0.3 * BM25KeywordOverlap). Scores are normalized to [0, 1]. If HybridScore > 0.05 or document is user-uploaded, the chunk is retrieved for synthesis.

Q12: Why did you choose a 70/30 weight ratio between Vector and BM25 search?

Answer: Empirical benchmarking showed that dense vectors provide superior conceptual and semantic retrieval (70%), but fail on exact technical codes, acronyms, and roll numbers (e.g. 23BTB0A33, JIRA-409, port 5432). The 30% BM25 keyword weight guarantees exact-match recall without semantic drift.

Q13: Why did you select Qdrant over ChromaDB, FAISS, or Pinecone?

Answer: Pinecone is a cloud-only SaaS which violates enterprise air-gapped privacy. FAISS is an in-memory index library without native persistence, metadata filtering, or REST APIs. ChromaDB has higher memory overhead. Qdrant is written in Rust, runs as a lightweight local container, supports payload metadata filtering, and offers fast cosine distance indexing.

Q14: How does extract_text_from_pdf in src/api.py handle scanned documents?

Answer: It implements a 2-stage pipeline: first, it attempts native text extraction via pypdf. If extracted text is 0 characters (scanned image), PyMuPDF (fitz) renders each PDF page to an in-memory PNG at 150 DPI, Base64-encodes it, and sends it to a local Ollama multimodal vision LLM with an OCR transcription prompt.

Q15: Why render PDF pages at 150 DPI instead of 300 DPI in PyMuPDF?

Answer: 150 DPI provides the optimal trade-off: it yields crisp, highly legible text for the vision LLM while keeping image payload size under 500KB per page, speeding up Base64 encoding and inference runtime by 4x compared to 300 DPI.

Q16: What embedding model is used, and what is its dimensionality?

Answer: We use nomic-embed-text served locally via Ollama embeddings endpoint (/api/embeddings). It generates 768-dimensional dense vector embeddings with an 8,192 token context window.

Q17: Why use RAG instead of fine-tuning an open-source LLM on enterprise data?

Answer: Fine-tuning bakes static knowledge into model weights: it cannot be updated dynamically when a document changes, risks hallucinating outdated policies, and provides 0 citation traceability. RAG allows instant knowledge updates via /api/upload, zero retraining cost, and 100% verifiable source document citations.

Q18: What is Citation Precision, and how did PrivAgent achieve 100%?

Answer: Citation Precision measures the percentage of generated factual claims that map directly to retrieved source documents. In evaluate.py, regex extraction verifies that every cited PDF, table, and Jira ticket exists in the raw investigator logs. With Critic auditing active, ungrounded claims are rejected, achieving 100% precision.

Q19: How does the system handle snake_case terms in document and ticket queries?

Answer: Worker agents normalize query tokens by replacing underscores with spaces (query.replace('_', ' ')) before computing BM25 token set intersections, ensuring queries like marketing_strategy_q2 correctly align with natural language text.

Q20: What happens if Qdrant vector database is down when docs_agent_node runs?

Answer: The node catches the Qdrant connection exception, logs a vector search fallback warning, and automatically falls back to sparse BM25 keyword matching over the in-memory document corpus without crashing.

Pillar 3: Cost Optimization, Semantic Kernel & Fallback Routing (Q21 - Q28)

Q21: How exactly does PrivAgent achieve a 90% cloud inference cost reduction?

Answer: Over 90% of routine enterprise requests (SQL generation, RAG retrieval, ticket lookups) are executed locally for free ($0 cost) using open-weights models (Qwen-2.5/Gemma) on local hardware via Ollama. Cloud Azure OpenAI (GPT-4o) is only invoked as a fallback when local models fail, time out, or hit complexity limits.

Q22: What is Semantic Kernel SDK and what role does it play in src/sk_router.py?

Answer: Semantic Kernel is Microsoft's enterprise AI orchestration framework. In PrivAgent, SemanticKernelRouter acts as an intelligent LLM gateway that encapsulates model connector configurations, handles primary-to-fallback failover policies, and tracks cumulative token savings.

Q23: How does the fallback timeout mechanism work in sk_router.py?

Answer: When query_local_ollama is called, urllib.request enforces a strict 12-second HTTP timeout. If Ollama is offline, hangs, or returns a 500 error, the exception is caught and execution instantly redirects to query_azure_openai.

Q24: How are cost savings calculated in real time in sk_router.py?

Answer: For every successful local query, approx_tokens = len(prompt.split()) + len(res_text.split()). It multiplies local tokens by $0.005 / 1k tokens (Azure GPT-4o reference price) and accumulates the total in self.estimated_cost_saved_usd.

Q25: What Azure OpenAI API version and deployment configuration are used?

Answer: It targets Azure REST API version 2024-02-01 with deployment gpt-4o, using api-key header authentication and private endpoint isolation.

Q26: Why use Ollama for local serving instead of raw Hugging Face Transformers?

Answer: Raw Transformers in Python lack continuous batching, memory optimization, and standardized REST endpoints. Ollama packages GGUF quantized models with llama.cpp C++ acceleration, exposing clean OpenAI-compatible endpoints with low memory footprint.

Q27: What happens if both local Ollama AND Azure OpenAI are unreachable?

Answer: sk_router logs an error message to state['logs'] ('Both Local & Azure OpenAI routes unavailable') and returns a safe fallback message ('[Agent] Processed default response. Query received.'), allowing the state machine to complete without a hard crash.

Q28: Why is Azure OpenAI chosen over public OpenAI API for enterprise fallbacks?

Answer: Azure OpenAI provides Microsoft enterprise SLAs (99.9%), guarantees that customer data is never used to train OpenAI models, supports private VNet connectivity, and complies with SOC2, HIPAA, and ISO 27001.

Pillar 4: Model Context Protocol (MCP) & Enterprise Tools (Q29 - Q35)

Q29: What is the Model Context Protocol (MCP) and why is it significant?

Answer: MCP is an open standard created by Anthropic that standardizes how AI applications expose tools, resources, and prompts to LLMs via JSON-RPC 2.0. It eliminates custom proprietary function-calling glue code and decouples agents from specific tool implementations.

Q30: Which MCP tools are registered in src/mcp_tools.py?

Answer: Five standard tools: mcp_query_database (Text-to-SQL), mcp_search_docs (Hybrid RAG), mcp_analyze_code (AST & Git grep), mcp_search_tickets (Jira ticket lookup), and mcp_web_search (Privacy-preserving web search).

Q31: What does the JSON-RPC 2.0 execution response look like in MCP?

Answer: {'jsonrpc': '2.0', 'result': {'tool': tool_name, 'status': 'success', 'mcp_compliance': True, 'data': result}, 'id': 1}. If an error occurs, it returns standard error codes like -32601 (Method not found).

Q32: How does db_agent_node execute Text-to-SQL securely?

Answer: It inspects table schema ('churn_data'), prompts the LLM for a SELECT query, strips markdown backticks, validates that the SQL starts with 'SELECT' (enforcing read-only access), and executes it via psycopg with column-name mapping.

Q33: How does code_agent_node search codebase repositories?

Answer: It extracts keywords from the task description, uses glob.glob to locate all .py files in src/ and root, and scans files line-by-line, returning matching filenames, line numbers, and code snippets.

Q34: How does ticket_agent_node filter Jira tickets in data/tickets.json?

Answer: It loads tickets.json, concatenates searchable text (title + description + component), and filters tickets based on word overlap with query keywords (>3 chars).

Q35: How does web_search_agent_node perform search without API keys?

Answer: It queries DuckDuckGo's HTML endpoint (html.duckduckgo.com/html/?q=...), parses result__a tags and result__snippet using regex, extracts direct destination URLs from uddg redirect parameters, and strips non-ASCII noise.

Pillar 5: Security, Governance, RBAC & HITL Safety (Q36 - Q42)

Q36: How does security_agent_node implement Role-Based Access Control (RBAC)?

Answer: It inspects user_query and user_role. If user_role != 'admin' and query contains restricted keywords (drop table, sudo, truncate, rm -rf, shutdown), it sets final_response = '[SECURITY BLOCK]...' and routes directly to END, preventing any tool execution.

Q37: How is Human-in-the-Loop (HITL) implemented in LangGraph?

Answer: When compiling the graph, workflow.compile(checkpointer=checkpointer, interrupt_before=['human_approval']) is set. In execution_router, if a task contains write keywords (update, delete, drop, insert) and approved is False, it routes to human_approval_node, where LangGraph automatically suspends graph state.

Q38: How does an administrator approve a paused HITL action?

Answer: The client sends POST /api/approve with thread_id and approved=true. The backend loads the checkpoint from PostgresSaver, updates state['approved'] = True, and calls graph.invoke(None, config) to resume execution from the exact interrupted state.

Q39: Where are database credentials and API secrets stored?

Answer: In infrastructure/.env, which is excluded from Git via .gitignore. get_env_val() dynamically loads values from system environment variables with fallback to .env.

Q40: How does PrivAgent prevent prompt injection attacks in investigator tools?

Answer: By isolating data retrieval from tool execution: retrieved database rows, ticket texts, and document chunks are strictly placed in data payloads and logs, never directly interpolated as executable instructions in system prompts.

Q41: What is the purpose of logs/traces.jsonl?

Answer: It provides an immutable, append-only compliance audit trail. Every agent execution, timestamp (UTC ISO format), thread_id, log message, and latency_ms is recorded for governance and auditing.

Q42: What security improvements would you recommend for production?

Answer: 1. Add JWT OAuth2 authentication middleware (Azure AD/Okta) on FastAPI endpoints. 2. Implement SQL AST validation via sqlglot to disallow multi-statement execution. 3. Add AES-256 encryption on stored PostgreSQL checkpointer blobs.

Pillar 6: System Design, Scaling, Failure Modes & Edge Cases (Q43 - Q50)

Q43: How would you scale PrivAgent to handle 1,000,000 daily active users?

Answer: 1. Replace synchronous HTTP execution with Celery/Redis asynchronous task queues and WebSocket log streaming. 2. Deploy a distributed vLLM GPU cluster behind an HAProxy load balancer with continuous batching. 3. Place PgBouncer in front of PostgreSQL for connection pooling. 4. Shard Qdrant collections on Kubernetes.

Q44: What was the empirical accuracy lift from the Critic Agent in evaluate.py?

Answer: In our 16-case benchmark suite, activating the Critic Auditing Layer increased aggregate response accuracy from 47.92% to 56.25% (an absolute +8.33% lift) with 100% citation precision.

Q45: What is the latency cost of running the Critic verification loop?

Answer: On CPU, running a second-pass LLM audit increases average query latency from 61.67s to 80.98s (+19.31s). In production with GPU-accelerated vLLM inference, this overhead drops to < 1.2 seconds.

Q46: How did you fix local model plan misclassifications in TC-10?

Answer: Lightweight models mapped codebase scans to sql_agent because prompts contained data words. I implemented an Order-Prioritized Plan Validator in planning_agent_node that checks for code signatures (.py, repo, function) before SQL keywords, achieving 100% accuracy on code routing.

Q47: How did the Critic solve the TC-14 hallucination trap?

Answer: When asked for a customer phone number missing from TICKET-101, baseline models hallucinated fake numbers. The Critic cross-referenced the draft against raw logs, detected the missing evidence, and forced the Analyst to state that phone numbers were unavailable (raising accuracy from 0% to 100%).

Q48: How does the in-memory QUERY_CACHE achieve sub-50ms latency?

Answer: In src/agents/router.py, QUERY_CACHE stores (normalized_query -> response). If a query matches an existing key and response > 30 chars, intent_agent_node fulfills it immediately with pathway='PATHWAY_CACHE', bypassing LLMs and tools entirely.

Q49: What happens if two concurrent requests use the same thread_id?

Answer: LangGraph's PostgresSaver uses optimistic concurrency control with row-level locks on checkpoint writes. In src/graph.py, a threading.Lock (_graph_lock) serializes graph compilation to prevent race conditions.

Q50: If an interviewer asks 'What is the biggest weakness of PrivAgent?', what is your answer?

Answer: Sequential investigator execution in Pathway 3 (tasks run one after another instead of in parallel). In production, I would use LangGraph's Send() API to execute independent investigator tasks (e.g. querying SQL and Docs simultaneously) in parallel, cutting multi-source query latency by 50%.